

Single Virtual function driver for current and future Intel Network devices

Anjali Singhai Jain

Jesse Brandeburg

Mitch Williams

Oct 2016, NetDev 1.2

Agenda

- Problem statement
- Evolution of SRIOV enabled Network Virtual function drivers
- Design Considerations
- Implementation
- Challenges
- Alternative Implementations
- Future Work
- The Take Away

In a perfect world…

1. Customers deploying our hardware devices would get forward and backward compatibility in the VM with our existing SRIOV enabled virtual function drivers.
2. The advanced features exposed, the performance enhancement etc. with next generation hardware would happen in a transparent fashion.
3. The low level software would be as sticky as possible so that the framework above it can last multiple generations.

The Holy Grail would be…

- Customers deploying mass scale Virtual Machine (VM) images for their Data center or Cloud would have a single stable Virtual Function driver that works on all of of Intel Network devices.
- And can be deployed from frozen VM images that work even when underlying hardware goes through upgrades.
- A single SRIOV enabled VF driver that works on all Intel present and future devices.

Whatever you do, don't change the VF driver!

Evolution of SRIOV Enabled Network VF drivers

- igbvf
  - Base connectivity
- ixgbevf
  - Improved performance and load balancing on multi-core systems with multiqueue support
- i40evf
  - VF driver that looks a lot like a PF driver in terms of its features support

All of these supported basic features such as

- TX and RX Checksum Offload
- Single Layer TSO

Introducing Base Compatibility Mode

Base mode supported **“Adaptive Virtual Function”** Driver

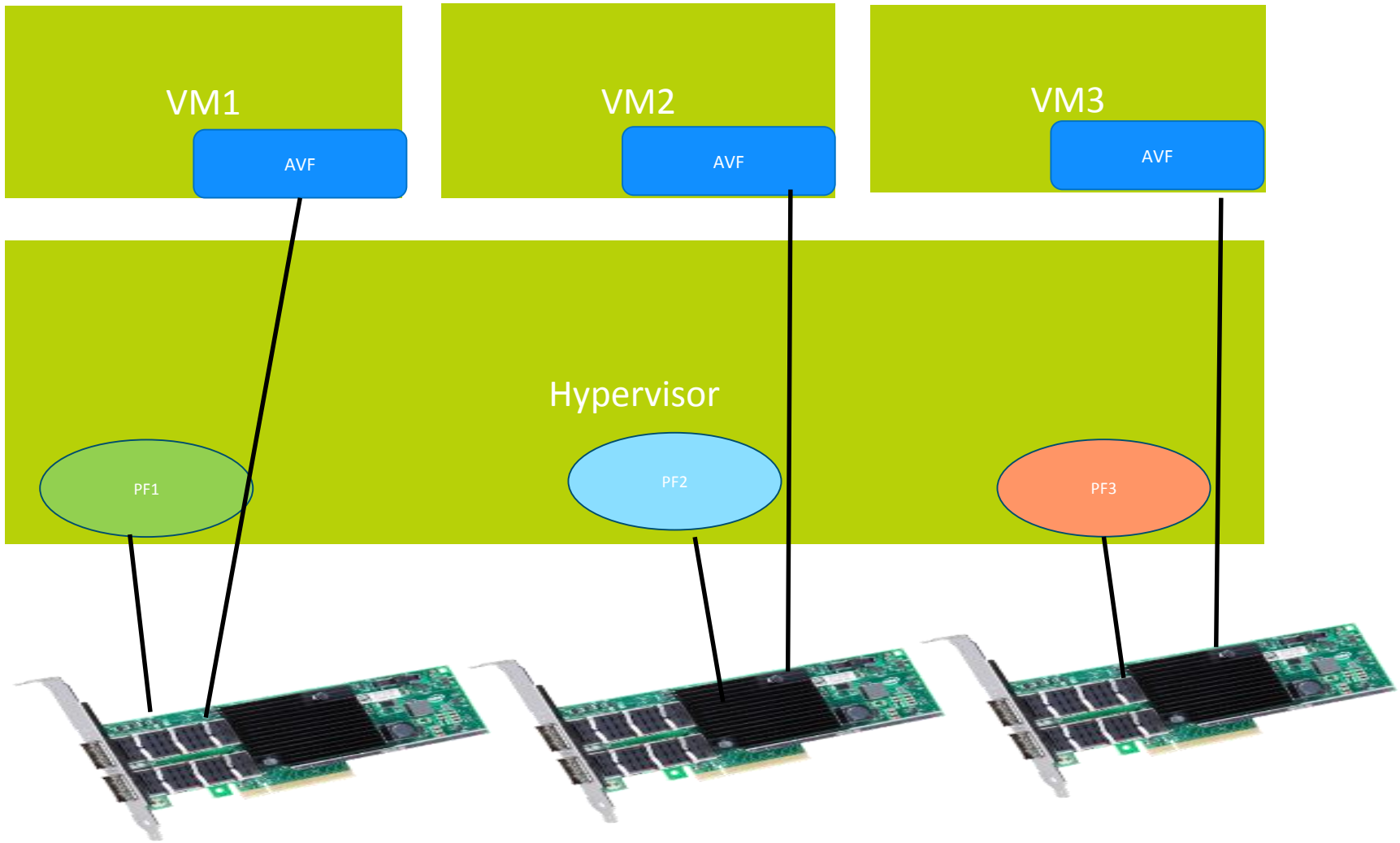
- **Defining Base Mode**
  - Single Device id
  - Support for single level checksum and TSO offload
  - Multiqueue Support
  - RSS

Preserving Base mode in Hardware and Software

- Fixed Minimum Register definition
- A fixed Meta data format for DMA
- A Hardware generic mailbox to talk to the PF
- A Software defined Virtual channel layered on top of Hardware mailbox for expansion

Leaving room for expansion

- A large range for hot path registers (Queue and Interrupt)
- Expandable Virtual channel capability negotiation

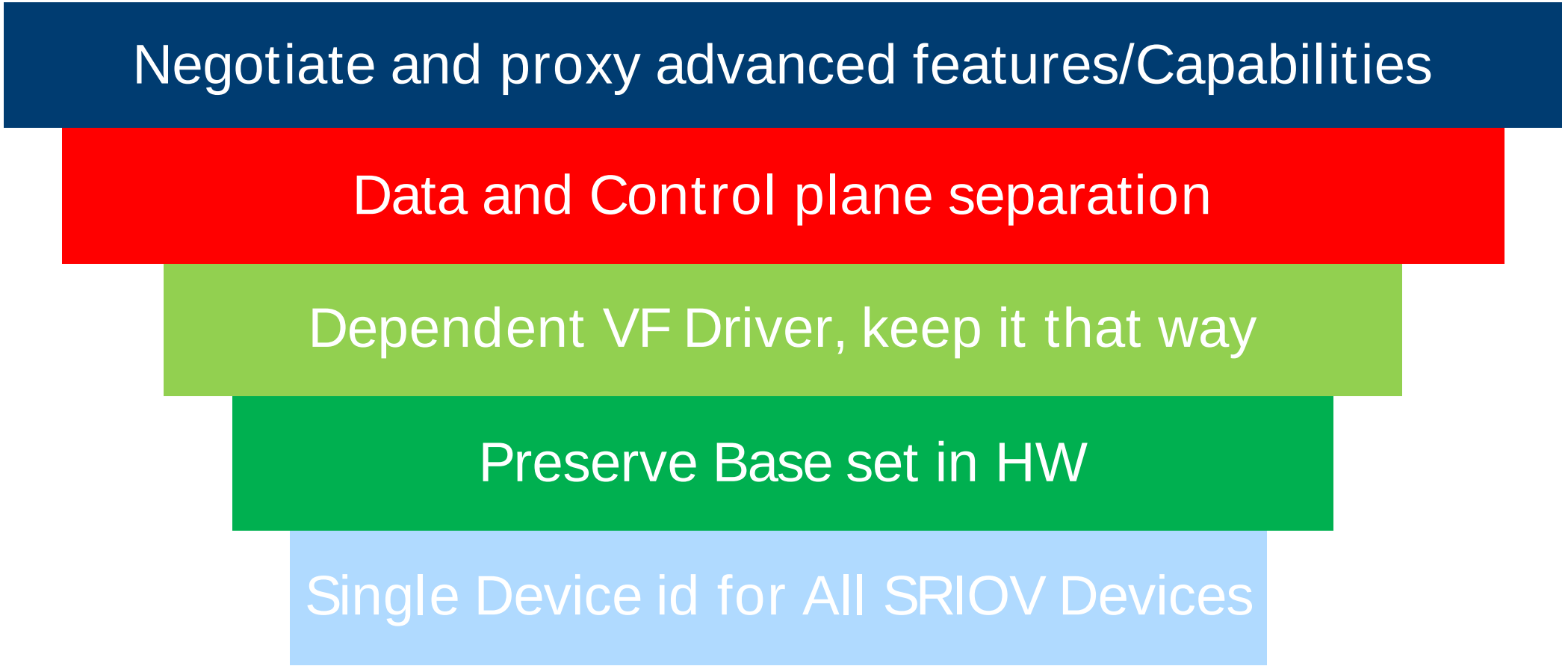


Adapter Gen1

Adapter Gen2

Adapter Gen3

Design Pattern



Implementation

- Published and shared Base register and DMA meta-data formats for all devices
- Published and shared Virtchannel between AVF and all PFs

Base features

- N number of Transmit Queues (at present 4, expandable to 128*)
- N number of Receive Queues (at present 4, expandable to 128*)
- N number of MSI-X vectors
- RSS configuration for Receive
- Fixed Receive and Transmit Descriptor format for checksum, TSO, Hash
- Known location for a basic set of registers including Transmit and Receive ring configuration and Head/Tail registers

Extended Features

- SW defined Virtchannel for Negotiation, on a HW Mailbox
- Negotiate and proxy advanced features with PF driver, features such as
  - VLAN trunking
  - Promiscuous Mode
  - Encapsulation Offload support
  - More filter resources and filter programming
  - RDMA
  - Larger RSS tables
  - Anything else needed for NFV, trusted VFs
- There is no specific limit to what a VF device can do in Hardware in the upcoming devices, but this model keeps the control and policy in PF driver

Challenges

- Keep the driver/devices from regressing away from base functionality
- Increase the validation matrix
- Every patch can potentially break the AVF model and make assumptions about underlying device and capability
- 0-day test bot

User Experience with new Model

- Always have at least basic connectivity out of the VM using any version of AVF
- The latest AVF drivers will keep adding to feature sets
- Current VM images can be used for new devices
- VM images need not be updated and verified for every new driver and driver change

Why not offload Virtio?

Strong desire to *make Virtio be efficient even when implemented in an I/O device*

- Examples: Enhanced ring structure, extensibility, meta-data, enhancements for event notification? capability discovery, control queue enhancements, vHost-pci
- Can Virtio 1.1 take these updates?
- vRing Enhancements
- Enable performance Optimization

Future work

Publishing a standards specification for an AVF device

Acknowledgements

- Parthasarathy Sarangam, Principal Engineer, Intel
- Donald E Wood, Software Architect, Intel
- Mitu Aggarwal, Senior Engineer, Intel

The Take Away

"Simplicity is the basis of all things beautiful."

- Reduce the direct access memory footprint for a SRIOV VF device to a bare minimum needed for hot path
- Deal with most urgent/pressing issues in the VF driver directly, otherwise punt it to PF driver
- Makes for a device and driver implementation that can be used for many generations of silicon